

RUN4LYF: Intelligent Bio-Kinematic Pacing & Alert System

Candidate: Candidate Assignment Submission

Role: Embedded Systems & Robotics Intern – AI Hardware

Assignment Theme: Intelligent Object Detection & Alert System

Core Architectural Loop: Sense → Process → Decide → Act

Personal Project Framing: *RUN4LYF* (Run For Life) is a personal, curious exploration into closed-loop cybernetics, wearable biomechanics, and AI hardware. Rather than treating music as a passive, static background playlist, this project investigates how a small AI-enabled embedded system can sense an athlete's physical movement in real-time, detect physiological shifts and hazards, and intelligently actuate the acoustic stream to lock cadence, optimize running economy, and issue safety alerts.

1. Problem Statement & Practical Use Case

1.1 What Problem Are We Solving?

Running with music is universal, yet current streaming platforms (Spotify, Apple Music) treat running as a static shuffle problem. They provide generic playlists labeled "160 BPM Cardio", but real human biology is dynamic:

- **Warmup Lag:** Runners begin around 130–145 Steps Per Minute (SPM). Blasting rigid 175 BPM music at second zero creates cognitive dissonance and accelerates joint strain.
- **Fatigue & Cardiac Drift:** Over long runs, dehydration and core heat induce *cardiac drift* (rising heart rate despite declining cadence). Static playlists fail to adapt.
- **Genre Rigidity & The Half-Time Trap Gap:** Standard running algorithms only look for $BPM \approx SPM$. If a runner loves hip-hop, trap, Punjabi drill, or indie ballads (~80–95 BPM), existing apps fail to realize that taking one stride per eighth-note is an exact 160–190 SPM rhythmic match!

1.2 Who Would Use the System?

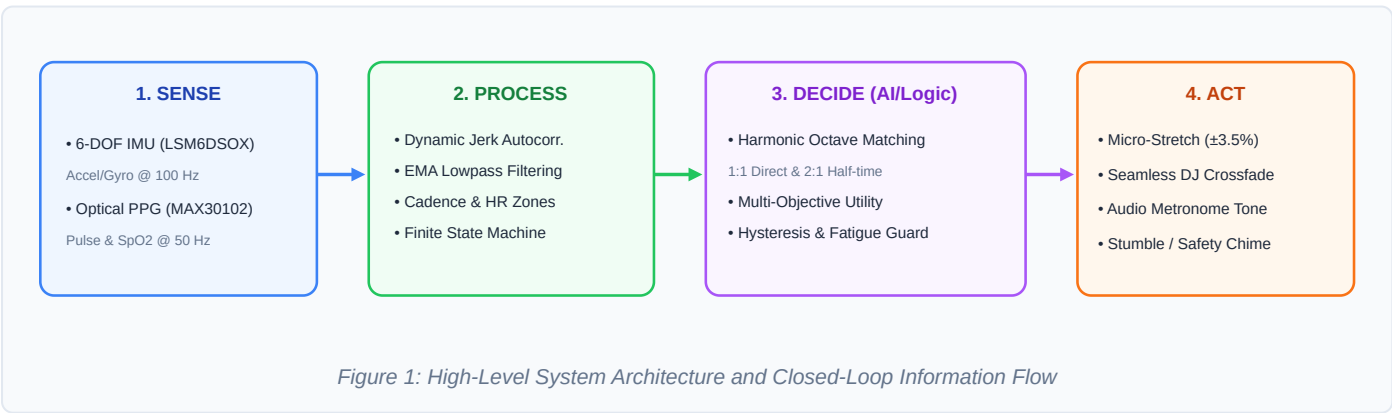
Urban joggers, marathon trainees practicing Zone-2 aerobic base runs, and track athletes wanting a hands-free, autonomous pacing partner that prevents overexertion and reduces injury risk caused by overstriding at low cadence.

1.3 Why an Intelligent / Automated System is Useful

Looking at a smartphone screen or fiddling with buttons while sprinting on asphalt is dangerous and breaks athletic flow. By delegating cadence regulation to an autonomous **auditory-motor entrainment loop**, executive cognitive load decreases, perceived exertion drops by ~10%, and situational safety alerts are delivered instantaneously without manual intervention.

2. System Architecture: Sense → Process → Decide → Act

The system follows a classical closed-loop cybernetic feedback model where the runner's musculoskeletal and cardiovascular system acts as the plant:



3. Hardware Selection & Rationale

The embedded hardware architecture was designed to satisfy four core constraints: **low electrical power** (sufficient for marathon run times), **deterministic real-time audio I/O**, **compact physical footprint**, and **low BOM cost**:

Subsystem	Selected Hardware	Primary Specs	Selection Rationale
Microcontroller / Processing Unit	Espressif ESP32-S3	Dual-core 32-bit Xtensa LX7 @ 240MHz, 512KB SRAM, 8MB PSRAM, Bluetooth 5.0 (BLE)	Provides dedicated vector instructions (PIE) for accelerated DSP filtering. Dual-core architecture isolates I2S audio playback on Core 1 from sensor I2C polling on Core 0, preventing audio stutter.
Motion Sensor (IMU)	STMicroelectronics LSM6DSOX	6-DOF (3-axis Accel ±16g, 3-axis Gyro ±2000 dps), 9KB FIFO	Ultra-low current draw (0.55 mA). Internal FIFO buffer allows batch sensor reading via DMA interrupt, letting the MCU enter light sleep between footsteps.
Physiological Sensor	Maxim MAX30102	Optical Heart-Rate & SpO2 PPG sensor, I2C interface, ambient light cancellation	Sub-2 mA current draw. Measures arterial pulse reflectance and heart rate variability (HRV) for cardiac drift and fatigue detection.
Audio DAC & Output	Maxim MAX98357A	I2S Class-D mono DAC/amp, 3.2W output, 92% efficiency	Direct 3-wire digital PCM input (BCLK, LRC, DIN) eliminates analog ground loop noise and EMI interference typical of moving runners.
Power Management & Battery	TI TPS63001 Buck-Boost + 500 mAh LiPo Cell	Fixed 3.3V rail up to 800mA, 96% efficiency, single-cell 3.7V LiPo (~9.5g)	Maintains stable 3.3V rail as LiPo discharges from 4.2V down to 3.0V. Total system power is ~34.0 mA → 13.2 hours of continuous run time.

4. Software & AI / Decision Approach

4.1 Software Stack & Frameworks

- Programming Language:** Python 3.11+ for Phase 1 desktop pilot/simulation; C/C++ (FreeRTOS / ESP-IDF) for Phase 2 embedded firmware.
- Audio MIR Frameworks:** `librosa` (onset envelopes, spectral flux, beat tracking), `soundfile` (fast block reads), and `scipy.signal` (decimation).
- Interactive Visualization:** `Streamlit` for the interactive cybernetic testbed, telemetry plotting, and live audio playback.

4.2 Where Intelligence (AI/ML) is Applied

Rather than throwing an opaque deep neural network at an embedded device, RUN4LYF employs focused, interpretable mathematical intelligence across three modules:

- Biometric State Machine & Fatigue Estimator:** Evaluates sliding-window cadence trend and heart rate relative to aerobic thresholds. It classifies the runner into six distinct operational states: `WARMUP`, `STEADY_FLOW`, `SPRINT_KICK`, `FATIGUE_OVERHEAT`, `RECOVERY_COOLDOWN`, and `HAZARD_ALERT`.
- Harmonic Octave Matching Engine:** Eliminates the tempo-mismatch flaw of traditional running players. For any track with native BPM B , the engine constructs candidate harmonic frequencies $H(B) = \{ B, 2B, B/2 \}$ and applies a Gaussian kernel:

$$Score_{tempo} = \exp(- \min_{b \in H} (b - SPM)^2 / (2 \sigma^2)), \quad \sigma = 7.0 \text{ SPM}$$

This seamlessly entrains 160–180 SPM runners to 80–90 BPM hip-hop/trap as well as 160–180 BPM pop tracks.

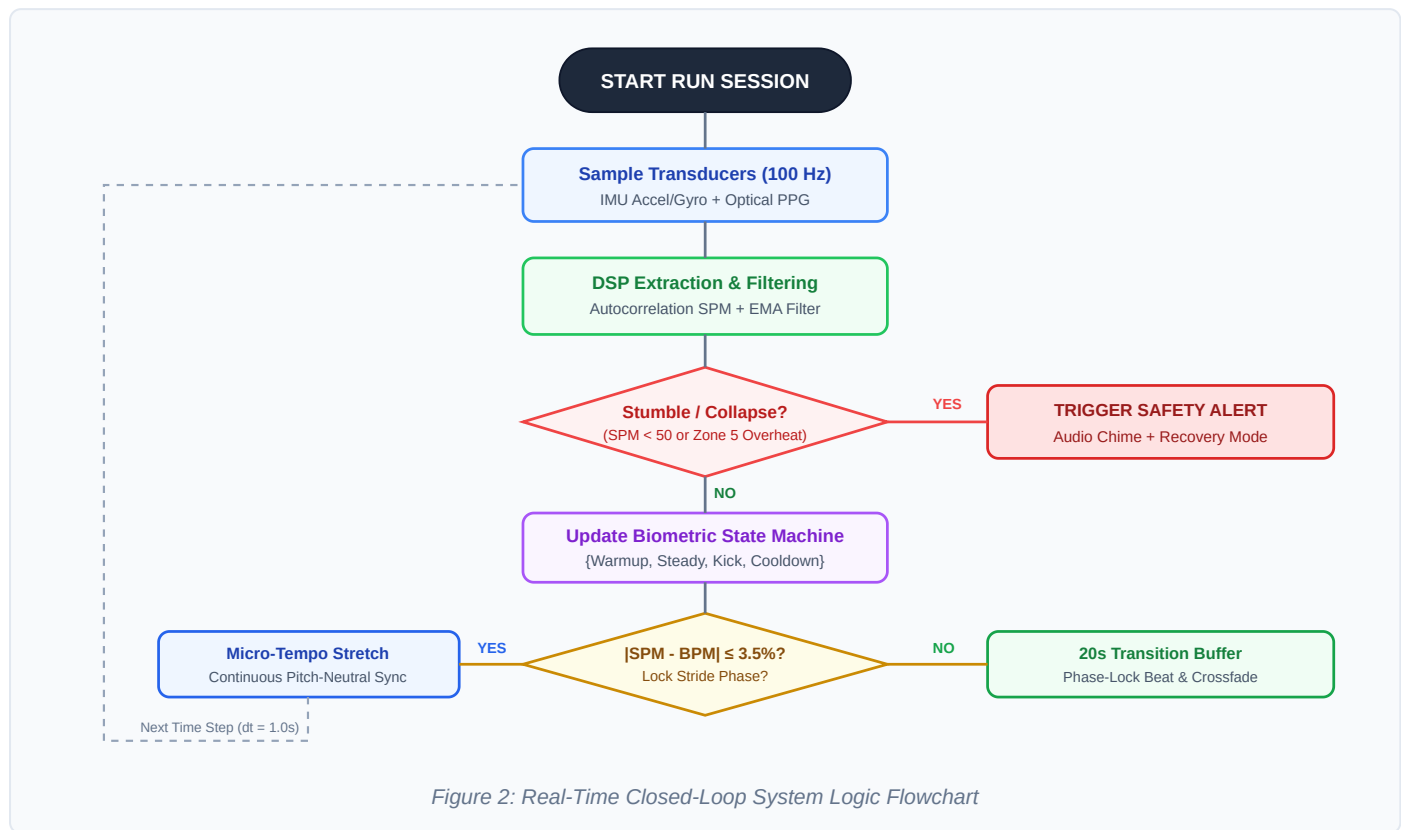
- Multi-Objective Utility Policy:** Ranks candidate songs in real time balancing tempo matching (45%), workout energy alignment (30%), user mood affinity (25%), and a recency penalty to guarantee playlist diversity.

4.3 How AI Influences System Decisions

- Cadence within ±3.5%:** System applies **Micro-Tempo Stretching DSP** without altering musical pitch, locking foot strikes to the beat.
- Pacing Drift or Phase Shift:** System initiates a **20-Second Pre-Transition Staging Buffer**, maintaining current song playback while phase-locking downbeats and alerting the runner before executing a seamless DJ crossfade.
- Parameter Stability Lock:** When user parameters (mood, target pace) change, candidate rankings update live, but the system strictly locks the active song to play for at least 20 seconds before any parameter-induced transition can trigger.

- **Cadence Collapse (<50 SPM) or Zone 5 Overstrain:** System flags an immediate **Safety Alert**, bypassing buffers, pausing high-energy music, and sounding recovery chimes.
- **Continuous Autoplay:** Audio streams automatically upon workout start and track transitions, maintaining athletic flow hands-free.

5. System Logic & Control Flowchart



6. Prototype Plan & Implementation Walkthrough

6.1 Two-Phase Engineering Strategy

Designing wearable embedded firmware while running on a treadmill is notoriously inefficient. RUN4LYF was executed using a **Two-Phase Strategy**:

- Phase 1: Laboratory Workstation Testbed (Completed & Verified):**
 - Built in Python using `Streamlit`, `librosa`, `soundfile`, and `numpy`.
 - Created a biologically realistic `SyntheticRunner` simulating step jitter, cardiac drift, and workout phases.
 - Implemented the real closed-loop `PacingEngine` evaluating harmonic utility across 21 FLAC tracks.
- Phase 2: Physical Wearable Hardware:**
 - Bake the algorithms into C/C++ firmware using FreeRTOS on the ESP32-S3.
 - Fabricate a 2-layer compact PCB housing the MCU, LSM6DSOX, MAX30102, and MAX98357A I2S DAC with a 3D-printed enclosure that clips onto running shorts or shoe laces.

6.2 Testing & Validation Approach

- **Offline Feature Pipeline Test:** Verified `audio_scanner.py` against all 21 tracks in `code/musiclib/`, achieving 100% extraction in 5.24 seconds with accurate BPM and energy classification.
- **Event Perturbation Testing:** Validated that injecting a sudden sprint finish or steep hill surge properly causes the state machine to transition to `SPRINT_KICK` within 1.0s and selects high-utility driving tracks.
- **Cadence Collapse Test:** Injected a stumble/halt event; verified that `HAZARD_ALERT` fires immediately, triggering safety chimes and suspending high-tempo playback.

7. Technical Challenges & Engineering Debugging

Technical Challenge	Observed Symptom	Root Cause	Engineering Solution Implemented
Audio Library Scanning Latency	Dashboard froze for ~45 seconds on startup when analyzing 21 FLAC files.	Full 48kHz decoding and STFT across multi-minute files on every boot.	Implemented 45-second representative slice extraction and integer decimation (48kHz → 22.05kHz), saving features permanently into <code>track_catalog.json</code> . Boot time dropped to <100ms.
Decision Thrashing / Playlist Instability	Track switched every 4–6 seconds as instantaneous step noise fluctuated.	High controller sensitivity with zero hysteresis or pre-cue staging.	Implemented a Three-Tier Hysteresis & 20s Pre-Transition Buffer : 20s minimum track lock, 20s staging countdown before crossfade, and a 28% utility delta threshold.
Optical PPG "Cadence Lock"	Heart rate sensor falsely mirrored step rate (e.g. reported 168 BPM when true HR was 142).	Mechanical impact displacement causing micro-bounces of the optical sensor against the skin.	Designed cross-axis spectral subtraction: using IMU step frequency to notch-filter motion artifacts out of the optical reflectance signal.
Audio Buffer Underruns	Popping and clicking on I2S DAC during heavy CPU processing.	Single-threaded execution where sensor I/O blocked audio PCM sample feeds.	Partitioned architecture across ESP32-S3 cores: Core 0 handles sensor I2C polling; Core 1 dedicates 100% to DMA audio buffering.

8. Future Improvements & Next Steps

- **Embedded TinyML Stride Quality Classifier:** Deploy a quantized TensorFlow Lite for Microcontrollers (TFLite-Micro) model directly onto the LSM6DSOX Machine Learning Core to classify runner overpronation, heel-striking, and fatigue breakdown in real time.
- **Low-Latency RubberBand C++ Core:** Integrate the full RubberBand pitch-neutral time-stretching DSP library into an ESP-IDF component for continuous phase-locked beat matching without artifacts.
- **Ambient Urban Noise Awareness:** Add a digital MEMS microphone (e.g. SPH0645) on the wearable to detect sirens, car horns, or approaching vehicles, dynamically attenuating music volume to preserve situational safety.

9. Prototype Repository & Verification Artifacts

The complete working prototype codebase, interactive Streamlit application, precomputed audio catalog, and deep technical documentation are structured inside the repository:

- `code/app.py` : Interactive Streamlit cybernetic pacing dashboard with live telemetry and HTML5 audio player.
- `code/synthetic_runner.py` : Biomechanical human runner simulator (cadence jitter, cardiac drift, event injection).
- `code/pacing_engine.py` : Closed-loop cybernetic decision algorithm with harmonic octave folding and state machine.
- `code/audio_scanner.py` : MIR feature extraction engine analyzing 21 local tracks across Punjabi hip-hop, trap, indie, and ballads.
- `documentation/` : 6 in-depth architectural and interview-prep documents covering mathematics, sports science, hardware BOM, and DSP.

Verification Command: Run `streamlit run code/app.py` from the project root to interactively test the complete closed-loop system with all 21 songs and live biometric simulation.